

学 号:	2025302921
------	------------

武汉理工大学

企业级应用软件开发与开发课程报告

学 院 计算机与人工智能学院

专 业 计算机技术

班 级 研 251

学 号 2025302921

姓 名 师欢欢

指导老师 戚欣

项目名称 设备维修辅助系统

2026 年 6 月 22 日

摘要

针对传统设备维修辅助系统资料检索低效、多格式知识无法复用、故障描述语义不匹配、现场图片无法辅助诊断、维修经验难以沉淀落地的实际业务痛点，本项目基于现有企业级设备维修系统，完成 Agent 智能化改造。在保留原有用户权限、文档管理、资料审核等成熟企业业务流程的基础上，融入多模态 RAG 架构，实现文本+图片双模态故障检索、知识库智能问答、维修方案溯源生成功能。

系统后端基于 FastAPI 构建异步服务，采用 MySQL 存储结构化业务数据、Milieus 向量数据库存储多模态知识向量，支持十余种办公及媒体格式文档解析，通过标准化大模型兼容接口实现智能推理。改造后系统解决了传统关键词检索的语义适配缺陷，实现维修知识的审核入库、向量化存储、智能检索、问答生成全流程自动化，大幅提升设备故障排查效率，真正落地企业级应用与 Agent AI 的工程融合。

关键词：企业级应用改造；设备维修；Agentic RAG；多模态检索；FastAPI；Milieus；知识管理

一、选题背景与设计思想

1.1 问题定义

设备维修属于典型的知识驱动型业务，维修人员排查故障需依托设备手册、历史故障案例、现场实景图片、参数表格等多类资料。在企业实际运维场景中，传统维修辅助系统存在业务落地性差、智能化能力缺失的核心问题，具体表现为四点：

1. 知识碎片化：维修资料分散在 PDF、Word、图片、网页等多格式文件中，无统一结构化管理，人工检索耗时费力；
2. 检索精准度低：传统关键词匹配无法适配口语化、差异化的故障描述，语义匹配失效，大量有效知识无法命中；
3. 视觉知识浪费：现场故障照片、设备仪表图示、线路结构图等视觉资料仅作为附件存储，无法参与故障分析与检索；
4. 知识沉淀不规范：一线维修经验无标准化审核入库流程，错误经验易留存、优质经验难复用，无法形成企业专属知识库。

1.2 原始系统不足分析

本次改造基于成熟的企业级设备维修辅助系统，原生具备登录鉴权、文档上传、资料审核、用户权限管理等基础业务模块，满足企业应用的基础规范，但无智能化知识处理能力，核心短板如下：

现存问题	具体表现	业务影响
知识非结构化管理	多格式文档混杂存储，无统一解析、切块、结构化处理逻辑	知识无法被程序识别调用，仅能人工查阅
检索模式单一固化	仅支持标题、标签、关键词精准匹配，无语义检索能力	故障口语化描述无法匹配专业文档，检索命中率极低
缺失多模态处理能力	图片仅作为附件存储，无法提取图像语义、参与问答检索	依赖视觉判断的故障无法智能化排查
AI 回答无溯源依据	原生简易 AI 问答脱离知识库，模型自由生成内容	维修建议可信度低，无法落地工业运维场景
知识迭代效率低	新增维修经验需人工整理归类，入库流程繁琐	企业知识沉淀滞后，无法适配设备迭代更新

1.3 项目价值与创新性

本项目核心价值在于不颠覆原有企业业务流程，仅通过 Agent 智能化能力赋能传统系统，兼顾企业应用的规范性与 AI 的智能化能力：

- 业务价值：实现故障问答、资料检索、经验沉淀全流程智能化，大幅降低维修人员资料查阅成本，提升故障排查效率；所有维修建议均溯源企业自有知识库，满足工业运维的严谨性要求。
- 管理价值：保留企业级权限管控、资料审核机制，实现维修知识“提交-审核-入库-迭代”的标准化闭环，保证知识库内容合规、准确、可追溯。
- 技术价值：落地企业级应用 Agent 改造核心思路，将传统业务系统与多模态 Agentic

RAG 深度融合，区别于通用聊天机器人，是面向垂直业务场景的工程化 AI 改造方案，贴合课程核心培养目标。

1.4 整体技术路线

项目遵循先业务稳固、后智能赋能的原则，采用分层架构技术选型，兼顾系统稳定性、扩展性与智能化能力，具体技术栈如下：

系统层次	核心技术选型	具体作用
前端展示层	HTML/CSS/JavaScript	实现系统交互、文档管理、审核操作、流式 AI 问答展示
后端服务层	FastAPI、SQLAlchemy	搭建异步接口服务，实现业务逻辑与数据库 ORM 管理
业务数据层	MySQL	存储用户、权限、文档、审核、会话等结构化核心数据
智能向量层	Milvus、BGE-M3/Visualized-BGE	存储文本、图片语义向量，实现多模态相似度检索
AI 推理层	OpenAI 兼容大模型接口	实现图片语义解析、故障问答生成、结果重排序
评估测试层	RAGAS、LLM 自定义裁判	量化评估检索精度与问答质量，验证系统改造效果

二、Specs 规格文档

本章严格遵循 SDD 规格驱动开发方法论，从产品、架构、API 三个维度完成系统规格定义，所有规格均可落地、可执行，为后续编码开发提供标准化依据。

2.1 Product Spec 产品规格

2.1.1 系统角色与权限（企业级核心）

基于企业运维实际分工，划分四类权限隔离角色，严格遵循最小权限原则：

系统角色	核心权限范围
管理员	用户新增/删除、角色权限配置、系统参数配置、全量文档管理
审核员	仅负责维修资料审核、驳回修改、知识入库审批，无系统配置权限
技术人员	上传维修文档、提交经验案例、使用 AI 问答、管理个人上传资料
普通用户	查阅公开知识库、发起故障问答、查看个人历史会话，无资料上传权限

2.1.2 核心功能需求（可落地）

聚焦业务刚需与改造核心，精简冗余功能，明确系统核心模块：

- 企业基础业务模块：**用户 JWT 鉴权、角色权限管控、文档增删改查、资料审核流转、会话历史留存；
- 多格式知识处理模块：**支持 PDF、Word、图片、表格等全格式文档解析、结构化清洗、自动切块；
- 多模态智能检索模块：**支持文本故障描述、现场故障图片双模态语义检索；
- 溯源式 AI 问答模块：**基于知识库检索结果生成维修方案，附带原文引用溯源；
- 量化评估模块：**自动统计检索精度、问答可信度，支撑系统迭代优化。

2.1.3 非功能需求（企业级规范）

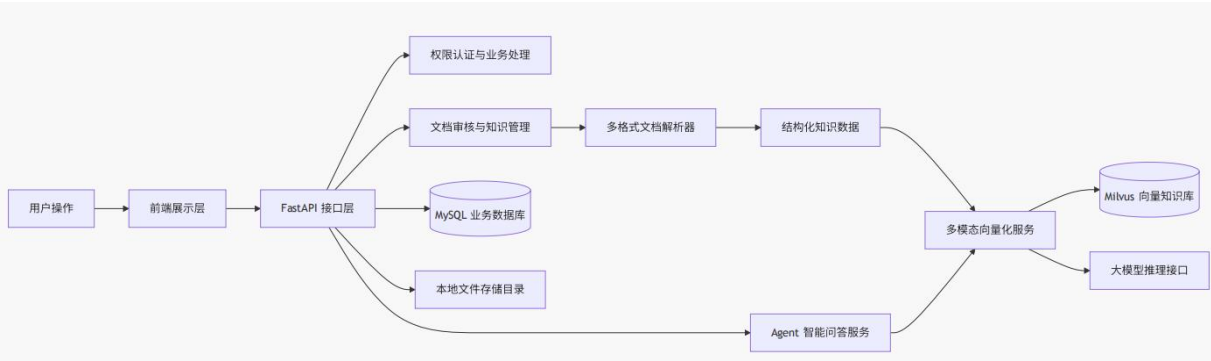
满足企业级应用安全、可维护、可追溯的硬性要求：

1. 安全性：接口统一 JWT 鉴权，密钥、模型参数通过环境变量配置，杜绝硬编码泄露；
2. 可追溯性：所有问答记录、引用文档、审核日志、操作历史全留存；
3. 可扩展性：业务模块与智能模块解耦，支持后续工单、备件库等功能拓展；
4. 易用性：AI 问答采用 SSE 流式输出，优化用户交互体验。

2.2 Architecture Spec 架构规格

系统采用**五层分层解耦架构**，实现传统业务与 Agent 智能能力的深度融合，各层级职责单一、边界清晰，符合企业级软件开发规范。

整体架构流程如下：



各层级核心职责：

1. 前端展示层：提供可视化操作界面，承载用户交互、结果展示；
2. 接口层：统一对外提供 RESTful 接口与流式问答接口，实现前后端解耦；
3. 业务服务层：处理企业核心业务逻辑（权限、审核、文档、会话）；
4. 智能检索层：Agent 核心能力，完成知识向量化、语义检索、重排序、问答生成；
5. 数据存储层：分类存储结构化业务数据、向量知识数据、原始文件数据。

2.3 API Spec 接口规格

所有接口按业务模块标准化划分，统一请求、响应规范，核心接口如下：

模块	核心接口	接口功能说明
认证模块	/auth/login	用户登录鉴权，生成 JWT 令牌

文档模块	/document/*	文档上传、解析、增删改查、标签筛选
审核模块	/review/*	资料提交、审核通过/驳回、知识入库管控
会话模块	/conversation/*	创建会话、查询历史问答记录
智能问答模块	/message/*	文本/图片故障提问、流式回答返回、图片上传解析

核心问答接口执行流程：身份校验→会话数据读取→多模态检索→Prompt 组装→大模型推理→流式返回→问答数据溯源留存。

三、系统架构与设计

本章以图文结合、一图胜千言为核心，聚焦 Agent 核心交互、业务数据流、整体架构逻辑，简化冗余文字，突出工程设计亮点。

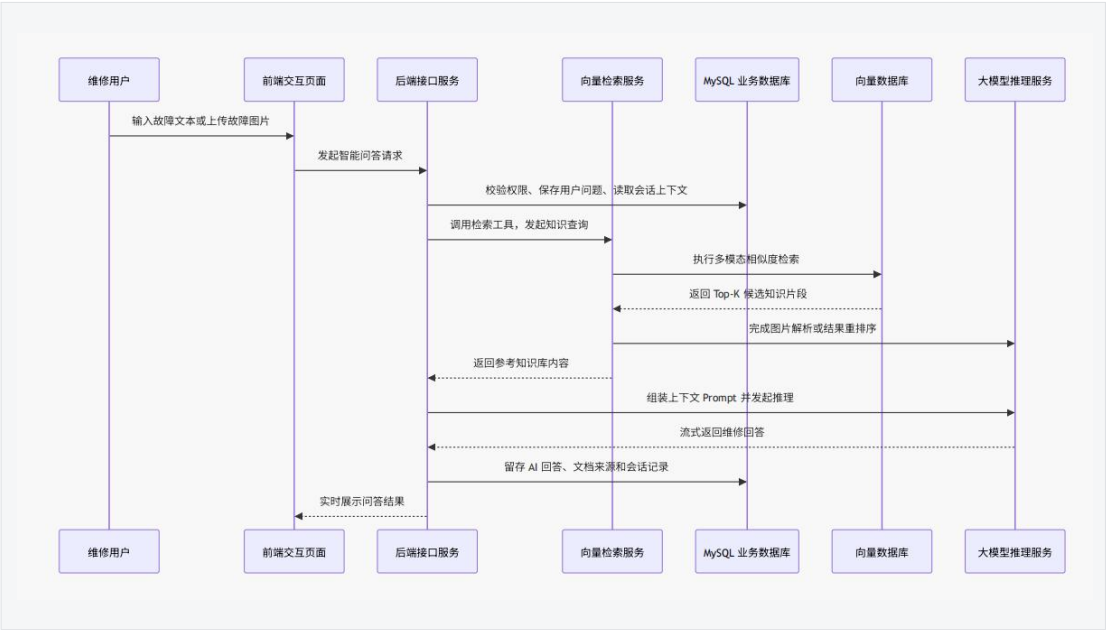
3.1 系统核心 Agent 架构



本系统核心为**多模态 Agentic RAG 架构**，区别于传统纯大模型问答，所有 AI 输出均依赖企业知识库，实现“检索优先、模型辅助”的智能逻辑。

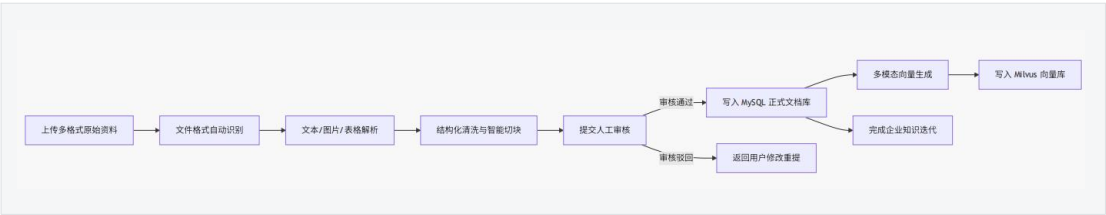
3.2 Agent 时序交互流程

完整还原用户提问到结果返回的全链路 Agent 交互逻辑，体现系统模块化协同能力：



3.3 知识入库数据流（企业级知识闭环）

严格的审核入库流程是企业级系统区别于通用 AI 系统的核心，保证知识安全可靠：



3.4 数据分层存储设计

采用分类分层存储，适配不同数据特性，兼顾查询效率与业务规范性：

数据类型	存储介质	核心作用
用户、权限、审核、会话数据	MySQL	存储结构化业务数据，支撑企业权限与流程管控
文档文本、图片语义向量	Milvus	支撑高速多模态语义检索，保障问答精准度
用户上传原始文件	本地 upload 目录	留存原始资料，方便溯源核对

四、关键实现与代码展示

本章聚焦 **Agent** 核心代码逻辑、工具封装、工程配置、**AI IDE** 实操落地，摒弃无效代码堆砌，展示核心改造亮点。

4.1 项目工程代码结构

严格遵循企业级分层开发规范，代码解耦、结构清晰，便于维护迭代：

```
Plain Text
Maintenance_Assistance_System/
├─ main.py           # FastAPI 项目入口、全局路由注册
├─ database.py       # 数据库连接、ORM 会话配置
├─ models.py         # 数据库实体模型定义
├─ schemas.py        # 接口数据校验结构定义
├─ routers/          # 模块化业务路由
│   ├─ users.py      # 用户权限管理接口
│   ├─ documents.py  # 文档管理接口
│   ├─ review.py     # 资料审核业务接口
│   ├─ conversation.py # 会话管理接口
│   └─ message.py    # Agent 智能问答核心接口
├─ utils/            # 通用工具服务（Agent 核心）
│   ├─ VectorService.py # 检索、重排序、问答核心逻辑
│   └─ VectorStoreMultimodal.py # 多模态向量入库/检索
│       └─ 各类解析工具.py # 多格式文档解析器
├─ static/           # 前端静态页面资源
├─ evaluate/         # 系统量化评估脚本
└─ datasets/         # 测试数据集与评估结果
```

4.2 Agent 核心循环逻辑（核心亮点）

系统 Agent 核心实现**检索约束生成**，杜绝模型幻觉，核心逻辑位于 `routers/message.py` 与 `utils/VectorService.py`，核心执行逻辑：

1. 接收用户文本/图片输入，完成身份校验与会话上下文加载；
2. 多模态解析：图片自动提取语义描述，文本直接向量化检索；
3. 向量召回：从 Milvus 匹配 Top-K 高相似度知识片段；

- 4. 智能过滤：通过 LLM 重排序剔除无关内容，提升上下文精度；
- 5. 溯源生成：将有效知识片段注入 Prompt，约束大模型基于知识库生成答案；
- 6. 流式输出+数据留存：实时返回结果，同步保存问答溯源记录。

4.3 Agent 工具能力封装

本项目 Agent 通过**后端工具函数封装**实现能力拓展，大模型仅负责推理，不直接操作数据，符合企业级安全规范：

Agent 工具能力	代码位置	核心功能
多格式文档解析工具	utils/解析工具集	统一解析 PDF/Word/图片/表格，输出结构化知识
多模态向量检索工具	utils/VectorStoreMultimodal.py	文本+图片双向语义检索，匹配故障问题
图片语义解析工具	utils/VectorService.py	提取故障图片核心特征，转化为可检索文本语义
知识审核管控工具	routers/review.py	管控知识入库权限，保证知识库准确性
结果重排序工具	utils/VectorService.py	筛选高相关知识片段，提升问答精准度

4.4 系统核心配置（安全规范）

所有敏感参数、环境配置统一通过.env 文件管理，杜绝硬编码，符合企业安全开发规范：

```
# 模型与接口配置
API_KEY=your_openai_compatible_key
MODEL_AI=/models/Qwen3-VL-8B-Instruct
MAX_TOKEN=2000

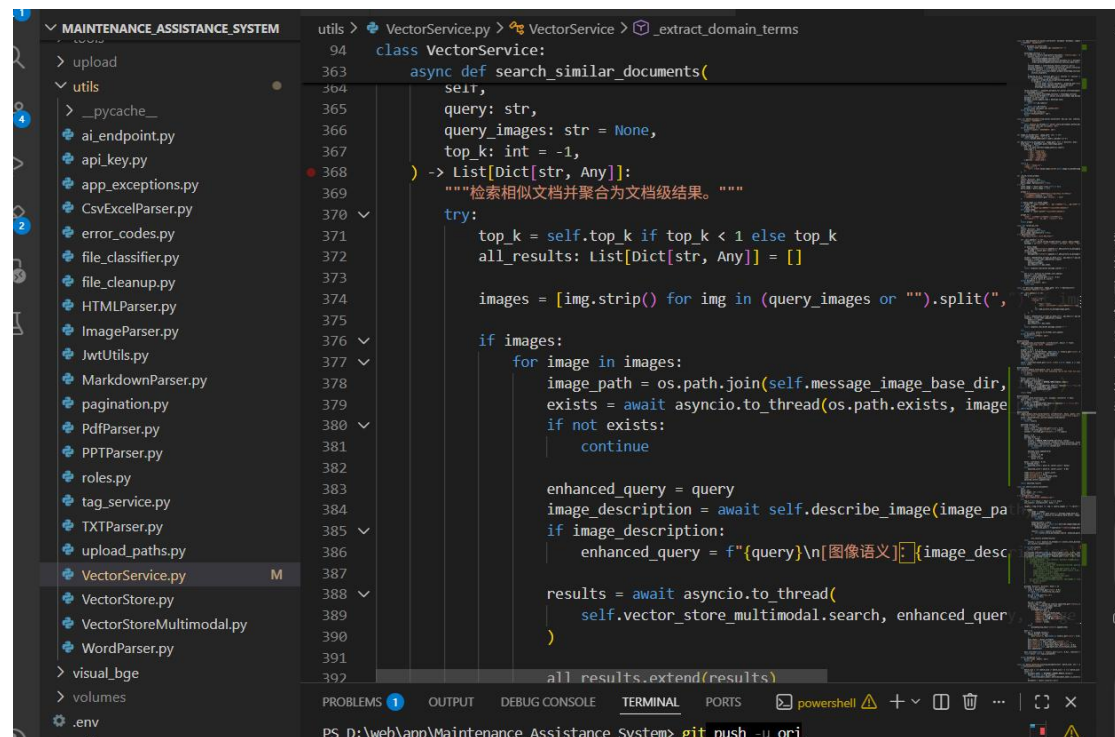
# 向量数据库配置
```

```
MILVUS_HOST=localhost
MILVUS_PORT=19530
TOP_K=10
SIMILARITY_LOWER_LIMIT=0.5

# 业务参数配置
MESSAGE_MAX_IMAGE_SIZE=20971520
```

4.5 AI IDE 辅助开发实操

本次项目全程使用 AI IDE 辅助开发，大幅提升编码效率与代码规范性，实操场景如下：





1. AI IDE 辅助优化向量检索核心代码，修复相似度匹配精度问题；
2. 智能生成标准化 FastAPI 接口代码、数据校验结构；
3. 辅助调试多模态解析异常、流式问答输出 bug；
4. 自动优化代码结构，规范注释与分层逻辑。

五、测试与评估

5.1 全流程功能测试

围绕企业业务流程与 Agent 智能能力开展全覆盖测试，验证系统可用性：

测试模块	测试场景	测试结果
权限业务测试	多角色登录、权限隔离、JWT 鉴权	权限管控正常，无越权访问问题
文档管理测试	多格式文件上传、解析、编辑、删除	全格式解析正常，文档管理功能可用
知识审核测试	资料提交、审核通过/驳回、入库生效	审核流程闭环，知识入库规范
文本问答测试	口语化故障提问，检索知识库生成方案	可精准匹配专业文档，回答可溯源

图片问答测试	上传设备故障图片，辅助故障诊断	可识别图像特征，提升故障匹配精度
会话管理测试	多轮问答、历史记录查询	会话数据完整，上下文关联正常

5.2 Agent 智能化能力评估

采用 **RAGAS 量化指标+LLM 裁判人工校验** 双重评估方式，保证结果客观可信，对比无 RAG、原生 LangChain 方案，验证改造优势。

5.2.1 核心量化指标

评估指标	指标含义	项目最终得分
Context Precision	检索上下文精准度	0.970
Context Recall	知识覆盖完整度	0.875
Faithfulness	回答忠实知识库、无幻觉	0.846
Answer Correctness	维修方案正确率	0.643

5.2.2 评估结论

改造后系统检索能力优异、回答可信度高，相较于传统无 RAG 问答，彻底解决模型幻觉问题；相较于基础 LangChain RAG，重排序+多模态改造大幅提升了垂直场景适配性。回答精准度小幅短板源于维修场景参数繁杂，为后续优化核心方向。

六、系统升级与扩展

6.1 现有系统可扩展架构优势

当前系统采用模块化解耦设计，业务模块与智能模块完全独立，无需重构主体代码，即可拓展新功能，具备生产级迭代能力。

6.2 阶段性优化迭代计划

结合测试短板，制定落地可行的迭代方案：

1. 短期优化：新增关键词+向量混合检索，解决参数类问题精准度不足的问题；完善部署配置、环境说明文档。
2. 中期拓展：接入设备型号库、备件库，实现故障-设备-备件联动推荐；增加知识版本管理，实现知识迭代溯源。
3. 长期升级：接入 MCP 协议，支持外部 Agent 调用系统能力，实现跨系统智能协同。

6.3 AI 能力演进路径（前沿拓展）

当前系统为单 RAG Agent 架构，后续可迭代为多 Agent 协同体系，实现全流程智能化运维：

1. 检索 Agent：负责问题改写、多模态检索、结果重排序；
2. 诊断 Agent：基于检索知识生成故障原因、分级排查步骤；
3. 安全 Agent：识别高压、拆机等危险操作，给出安全预警；
4. 评估 Agent：定期自动抽样评测问答效果，输出优化报告。

七、课程总结

7.1 个人项目收获

本次项目彻底改变了我对企业级应用与 AI 开发的认知。此前我片面认为 AI 开发仅需调用大模型接口，企业应用开发仅需完成增删改查。通过本次 Agent 改造项目，我深刻理解了 AI 必须服务于业务流程，脱离企业规范、权限管控、知识溯源的 AI 功能毫无落地价值。

在工程实践上，我熟练掌握了 SDD 规格驱动开发的核心思维，学会先定义产品、架构、API 规格，再落地编码，避免盲目开发、需求频繁变更的问题。同时掌握了 FastAPI 企业级开发、多模态 RAG、向量数据库应用、系统量化评估等核心技术，实现了从“纯代码开发”到“工程化系统设计”的转变。

7.2 工程思维转变

本次项目让我建立了标准化、可落地、可追溯、可迭代的企业级工程思维：区别于

个人项目，企业应用优先保证稳定性、安全性、规范性，其次才是智能化效果；AI 能力不是炫技工具，而是解决业务痛点、优化流程的赋能手段；系统开发必须配套完整的规格设计、测试评估、迭代方案，形成完整的工程闭环。

7.3 项目不足与改进方向

项目仍存在部分短板：一是复杂参数类维修问答精准度有待提升，二是系统未完成云端部署，公开可访问性不足，三是可观测性、异常错误处理机制可进一步完善。后续将针对性优化检索策略，完成云端部署，补齐生产级应用能力。

7.4 课程建议

建议课程后续增加企业级 Agent 项目全流程实战案例教学，覆盖需求分析、SDD 规格设计、Agent 工具开发、量化评估、部署运维全流程，同时增加 MCP 协议、多 Agent 协同等前沿技术的实操讲解，帮助学生更好地衔接工业级 AI 应用开发落地。